

Casa di Cura Misericordia Navacchio S.r.l. — IT

Cruscotto Degenze

Prompt ufficiale di sviluppo

Documento **Specifica per lo sviluppo con assistente AI**

Versione **1.0 — 16/09/2026**

Stack **Python 3.12 · Flask · SQLAlchemy · ReportLab**

Fase 1 **Versione locale (SQLite)**

Fase 2 **VM Windows Server nel dominio cdcmisnav.local (SQL Server)**

Riferimento **PrototipoCruscotto_Degenze_v5.html**

Redatto da **Micky — IT**

Come usare questo prompt

- Incolla tutto il testo che segue, dalla sezione 1 alla sezione 12, come primo messaggio all'assistente di programmazione AI (per esempio Claude Code).
- Allega il file `PrototipoCruscotto_Degenze_v5.html`: è il riferimento grafico e funzionale.
- Lavora una fase e un passo alla volta. Non chiedere mai "fai tutto": a ogni passo verifica il risultato e solo dopo vai avanti.
- La versione copiabile di questo testo è nel file `Prompt_Cruscotto_Degenze.md`.

1. Ruolo e modo di lavorare

Sei uno sviluppatore Python senior esperto di applicazioni web Flask in ambito sanitario. Lavori insieme a Micky, amministratore IT junior della Casa di Cura Misericordia Navacchio, al suo primo progetto di sviluppo reale. Il tuo compito è costruire con lui l'applicazione **Cruscotto Degenze** descritta in questo documento, prima in versione locale e poi in versione di rete su una macchina virtuale.

Regole di lavoro, da rispettare sempre:

- Procedi per **piccoli passi numerati**. Alla fine di ogni passo fermati, riassumi cosa hai fatto, spiega come verificarlo (comando da lanciare, pagina da aprire, risultato atteso) e attendi conferma prima di continuare.
- Spiega in **italiano semplice** e con **esempi concreti** ogni concetto nuovo (per esempio: cos'è una migrazione, cos'è un blueprint, perché si usa l'hash della password).
- Se un requisito è ambiguo o in conflitto con il prototipo, **chiedi prima** di scrivere codice.
- **Non aggiungere funzioni** non elencate qui. Se ne ritieni utile una, proponila e aspetta l'approvazione.
- **Non modificare file esistenti** senza dirlo: per ogni modifica indica il file e mostra cosa cambia.
- Nomi di dominio in italiano (`paziente`, `nota`, `reparto`, `dimissione`); convenzioni tecniche standard in inglese dove sono d'uso (`models.py`, `routes.py`, `create_app`).
- Commenti che spiegano il **perché**, non il cosa. Stile PEP 8.

2. Contesto

La Casa di Cura Misericordia Navacchio è una struttura sanitaria privata con tre reparti: **AOUP**, **Setting 1 — Low Care 1** e **Setting 2 — Low Care 2**. Il Cruscotto Degenze è un'applicazione web interna, raggiungibile solo dalla rete aziendale. Serve a:

- vedere i pazienti ricoverati in ogni reparto, ordinati per data presunta di dimissione;
- tenere per ogni paziente un'**Agenda** di note organizzative scritte da medici, infermieri e amministratori;
- seguire una **checklist di dimissione** di 20 voci;
- scaricare un **PDF** riepilogativo del paziente da stampare.

Il Cruscotto **non è una cartella clinica**: i dati clinici restano nel gestionale ResMedica. Si registrano solo i dati minimi necessari: nome, cognome, posto letto, date, gravità e note organizzative.

Il prototipo allegato (`PrototipoCruscotto_Degenze_v5.html`) è il riferimento per aspetto grafico, testi, colori e comportamento: tutto ciò che fa il prototipo va replicato. Dove il prototipo e questo documento non coincidono, **vale questo documento**.

3. Stack tecnologico (vincolante)

- **Python** 3.12 o superiore.
- **Flask** 3.x con *application factory* (`create_app`) e *blueprint* separati per area.
- **Flask-SQLAlchemy** come ORM e **Flask-Migrate** (Alembic) per le migrazioni del database.
- **Flask-Login** per le sessioni e **Flask-WTF** per form e protezione CSRF.
- **Password**: solo hash, con `werkzeug.security.generate_password_hash` e `check_password_hash` (algoritmo predefinito scrypt).
- **Interfaccia**: template Jinja2 con il CSS del prototipo; JavaScript semplice solo dove serve (salvataggi immediati, finestre). Niente framework frontend, niente fase di build, **nessuna risorsa da CDN**: tutto servito dall'applicazione.
- **PDF**: generati lato server con **ReportLab**, con il font DejaVu Sans incluso nel progetto per gli accenti.
- **Database**: **SQLite** nella versione locale; **Microsoft SQL Server** (istanza su `sql-misnav`) nella versione di rete, tramite pyodbc e *ODBC Driver 18 for SQL Server*. Lo stesso codice deve funzionare con entrambi cambiando solo la variabile `DATABASE_URL`. Per i test usa i tipi `Unicode` / `UnicodeText` di SQLAlchemy, così SQL Server salva correttamente le lettere accentate.
- **Server applicativo** in produzione: **Waitress**, dietro **IIS** come reverse proxy HTTPS.
- **Configurazione** tramite file `.env` letto con `python-dotenv`. Nessuna password o chiave nel codice. Fornisci un `.env.example` commentato.
- **Test** con `pytest`. Dipendenze con versioni esatte in `requirements.txt`. Controllo versione con **Git**.

4. Ruoli e permessi

I ruoli sono cinque, con questi codici: `ADMIN` (Amministratore), `MEDICO` (Medico), `INFERMIERE` (Infermiere), `OSS` (OSS) e `AMMIN` (Amministrazione / IT).

Cosa può fare ciascun ruolo:

- **Vedere** home, Agenda e checklist: tutti i ruoli.
- **Scrivere note** in Agenda: ADMIN, MEDICO, INFERMIERE.
- **Modificare o eliminare una nota**: solo l'autore della nota. Nemmeno l'ADMIN può modificare o eliminare note scritte da altri.
- **Spuntare la checklist**: ADMIN, MEDICO, INFERMIERE.
- **Cambiare la gravità**: ADMIN, MEDICO, INFERMIERE.
- **Inserire un paziente**: ADMIN, MEDICO, INFERMIERE.
- **Modificare la data presunta di dimissione**: ADMIN, MEDICO, INFERMIERE.
- **Eliminare un paziente**: solo ADMIN.
- **Scaricare il PDF**: tutti tranne OSS.
- **Gestione utenti e registro accessi**: solo ADMIN.
- **Cambiare la propria password**: tutti.

Regole di implementazione:

- I permessi stanno in **un solo punto** (`app/permessi.py`), come dizionario ruolo → permessi, con un decoratore `@richiede_permesso("...")` per le route.
- **Ogni permesso va verificato sul server**. Nascondere un pulsante non basta: una richiesta non autorizzata risponde `403` con una pagina in italiano.

5. Requisiti funzionali

5.1 Accesso

- Accesso con nome utente e password. **Non esiste registrazione autonoma**: gli account li crea l'ADMIN e consegna le credenziali agli interessati.
- Al primo accesso, e dopo ogni reimpostazione, l'utente **deve cambiare la password** prima di usare il programma. Requisiti della nuova password: almeno 8 caratteri, almeno una lettera e un numero, diversa da quella temporanea.
- Dopo **5 tentativi falliti** consecutivi l'account resta bloccato per 15 minuti. Il messaggio di errore è sempre generico ("Nome utente o password non corretti"), così non si rivela se un utente esiste.
- Un utente disattivato non può accedere.
- Logout manuale, e **logout automatico dopo 15 minuti di inattività** (valore in `.env`), perché i PC di reparto sono condivisi.
- In alto a sinistra la barra laterale mostra nome, cognome e ruolo dell'utente collegato, come nel prototipo.
- Il primo amministratore si crea da terminale con il comando `flask crea-admin`, che chiede nome, cognome, nome utente e password.

5.2 Home "Pazienti ricoverati"

- Tre riquadri, uno per reparto, in quest'ordine: AOUP, Setting 1 — Low Care 1, Setting 2 — Low Care 2. Ogni riquadro mostra il numero di pazienti ("3 pazienti", "1 paziente", "nessun paziente").
- Colonne: **Paziente** (Cognome Nome), **Letto**, **Data arrivo**, **Dimissione presunta**, **Gravità**, **Azioni**.
- Si vedono solo i pazienti con stato `RICOVERATO`.
- **Ordinamento** dentro ogni reparto:
 - prima per data presunta di dimissione, dalla più vicina alla più lontana;
 - i pazienti senza data vanno in fondo e mostrano "da definire";
 - a parità di data si ordina per letto in ordine naturale (A-2 prima di A-10).
- Legenda della gravità: Critico (rosso), Instabile (giallo), Stabile (verde), Non valutato (pallino tratteggiato).
- **Gravità**: il pallino è cliccabile per chi ha il permesso e ruota nel ciclo non valutato → verde → giallo → rosso → non valutato. Il salvataggio è immediato. Per gli altri il pallino è disabilitato.
- **Dimissione presunta**: chi ha il permesso può modificarla cliccando sulla data. Si apre un piccolo form con un campo data, che si può anche svuotare. La data non può essere precedente all'arrivo. Dopo il salvataggio l'ordinamento si aggiorna.
- **Azioni**:
 - **Agenda**, con il numero di note visibili del paziente;
 - **Scarica PDF**, se l'utente ha il permesso;
 - **Elimina**, solo per l'ADMIN.
- Pulsante **Inserisci paziente** nell'intestazione di ogni reparto, per chi ha il permesso.
- Se un reparto è vuoto, mostra "Nessun paziente ricoverato in questo reparto".

5.3 Inserimento paziente

- Campi: **Nome***, **Cognome***, **Posto letto***, **Data di arrivo*** (predefinita: oggi), **Data presunta di dimissione** (facoltativa). Limiti: nome e cognome al massimo 80 caratteri, letto al massimo 10. Gli spazi iniziali e finali vengono tolti.
- La dimissione presunta, se indicata, non può essere precedente all'arrivo.
- Il letto non può essere già occupato da un paziente **RICOVERATO** dello stesso reparto. In quel caso il messaggio è, per esempio: "Il letto A-01 risulta occupato da Rossi Mario".
- Un nuovo paziente parte con gravità "non valutato" e checklist vuota.

5.4 Eliminazione paziente

- Solo ADMIN, dopo una finestra di conferma che spiega le conseguenze.
- L'eliminazione è **logica**: `stato = ELIMINATO` e vengono registrati chi e quando. Il paziente sparisce dal cruscotto e il letto torna libero; note, versioni e checklist restano nel database.
- **Nessuna cancellazione fisica** di pazienti o note, in nessun punto del programma.

5.5 Agenda

- Si apre come finestra modale sulla home. L'intestazione riporta Cognome Nome, reparto, letto, data di arrivo e dimissione presunta.
- Il contenuto è disposto in quest'ordine: **checklist** in alto, poi il campo per la **nuova nota**, poi **l'elenco delle note**.
- **Ordine delle note**: dalla più recente alla più vecchia. A parità di data viene prima quella inserita dopo (`data_nota` decrescente, poi `id` decrescente). Le note sono raggruppate per giorno sotto un'intestazione, per esempio "Oggi, 16/09/2026" oppure "15/09/2026".
- **Ogni nota** mostra:
 - l'autore con il ruolo, per esempio "**Medico**: Mario Rossi" oppure "**Infermiere**: Carlo Bianchi", con il ruolo nel colore del prototipo;
 - la data in formato gg/mm/aaaa, **senza ora**;
 - "modificata il gg/mm/aaaa", se la nota è stata modificata;
 - il testo, conservando gli a capo.
- **Nuova nota**:
 - sopra il campo compare la firma, per esempio "Scrivi come Infermiere: Carlo Bianchi, data 16/09/2026";
 - il testo è obbligatorio, al massimo 2000 caratteri;
 - nome, cognome e ruolo dell'autore vengono **copiati nella nota** al momento della scrittura: se in futuro l'utente cambia ruolo, la nota resta firmata come prima.
- **Modifica ed Elimina**: i pulsanti compaiono, e le operazioni sono consentite dal server, solo per l'autore.
 - *Modifica*: il testo precedente viene salvato in `nota_versione` e la nota riceve la data di modifica. Il testo non può restare vuoto.
 - *Elimina*: eliminazione logica, previa conferma. La nota sparisce da Agenda e PDF ma resta nel database, con chi l'ha eliminata e quando.
- Chi non può scrivere (OSS, AMMIN) vede un messaggio di sola lettura al posto del campo.
- Il contatore delle note sul pulsante Agenda in home si aggiorna dopo ogni operazione.

5.6 Checklist di dimissione

- Le voci stanno nella tabella `voce_checklist`, ognuna con un codice fisso, un testo e un ordine. Il testo si può cambiare senza perdere le spunte, perché le spunte sono legate al codice. Contenuto iniziale:

DIMISSIBILE	Paziente dimissibile
ESAMI	Esami completati
REFERTI	Referti acquisiti
TERAPIA	Terapia definita
PRESCRIZIONI	Prescrizioni predisposte
LETTERA_PRED	Lettera di dimissione predisposta
LETTERA_FIRMA	Lettera firmata
CATETERE	Catetere: definito mantenimento/rimozione
MEDICAZIONI	Medicazioni programmate
AUSILI	Ausili prescritti
PALLIATIVISTI	Palliativisti attivati se necessari
CONTINUITA	Continuità assistenziale attivata
DATA_DIM	Data dimissione concordata
FAMILIARI	Familiari avvisati
TRASP_DEFINITO	Trasporto necessario definito
TRASP_RICHIESTO	Trasporto richiesto
TRASP_CONFERMATO	Trasporto confermato
DOCUMENTAZIONE	Documentazione consegnata
FARMACI	Farmaci consegnati/prescritti
EFFETTI	Effetti personali consegnati

- Riquadro:** richiudibile, intitolato "Checklist dimissione", con contatore (per esempio 5/20) e barra di avanzamento. Il contatore diventa verde a 20/20. Lo stato aperto/chiuso del riquadro viene ricordato dal browser (`sessionStorage`).
- Voci:** disposte su due colonne. Ogni spunta si salva subito (richiesta `fetch` con token CSRF) senza ricaricare la pagina.
- Nessuna tracciatura:** non si registra chi ha spuntato né quando.
- Sola lettura:** per OSS e AMMIN le caselle sono disabilitate, e il server rifiuta comunque la modifica.

5.7 PDF del paziente

- Generato lato server con ReportLab, formato A4, scaricato direttamente (`Content-Disposition: attachment`). Nome del file: `Agenda_Cognome_Nome_AAAA-MM-GG.pdf`.
- Contenuto**, in quest'ordine:
 - titolo "Scheda paziente e Agenda" e nome della struttura;
 - dati del paziente: paziente, reparto, posto letto, gravità, data di arrivo, dimissione presunta ("da definire" se manca), "Documento generato il gg/mm/aaaa da Ruolo: Nome Cognome";
 - checklist su due colonne, con caselle spuntate o vuote e contatore;
 - storico delle note visibili, nello stesso ordine e con la stessa firma dell'Agenda, compresa l'eventuale "modificata il";
 - piè di pagina con "Cognome Nome - letto X" e "Pagina x di y".
- Il download viene registrato nel registro accessi.

5.8 Gestione utenti (solo ADMIN)

- Elenco utenti:** nome, cognome, nome utente, ruolo, stato (attivo, disattivato, bloccato) e ultimo accesso.

- **Nuovo utente:** nome, cognome, nome utente, ruolo e password temporanea (almeno 8 caratteri). Il nome utente è univoco senza distinzione tra maiuscole e minuscole. Al primo accesso l'utente è obbligato a cambiare la password.
- **Operazioni su un utente esistente:**
 - disattivare o riattivare, ma non il proprio account;
 - cambiare il ruolo;
 - reimpostare la password (nuova password temporanea, con obbligo di cambio);
 - sbloccare un account bloccato.
- **Nessuna cancellazione** di utenti: le note devono sempre poter risalire al loro autore.
- **Il mio profilo:** ogni utente vi trova il cambio della propria password.

5.9 Registro accessi (solo ADMIN)

- Tabella `evento_audit` in sola aggiunta: gli eventi non si modificano e non si cancellano dall'applicazione.
- **Eventi registrati:** `LOGIN_OK`, `LOGIN_FALLITO`, `LOGOUT`, `PAZIENTE_CREATO`, `PAZIENTE_ELIMINATO`, `DIMMISSIONE_MODIFICATA`, `GRAVITA_MODIFICATA`, `NOTA_CREATA`, `NOTA_MODIFICATA`, `NOTA_ELIMINATA`, `PDF_SCARICATO`, `UTENTE_CREATO`, `UTENTE_MODIFICATO`, `PASSWORD_REIMPOSTATA`, `PASSWORD_CAMBIATA`.
- **Dati di ogni evento:** data e ora, utente, azione, tipo ed id dell'oggetto, un breve dettaglio e l'indirizzo IP del client. Il dettaglio **non contiene mai il testo delle note**.
- **Pagina di consultazione:** filtri per periodo, utente e azione, con paginazione.

6. Modello dati

I tipi sono indicativi e SQLAlchemy li traduce per SQLite e SQL Server. Le date e ore si salvano in **UTC** e si mostrano in ora italiana (Europe/Rome). `data_nota` è la data italiana del giorno in cui la nota è stata scritta.

reparto			
id	PK intero		
codice	testo(10)	univoco	es. A0UP, S1, S2
nome	testo(60)		es. "Setting 1 – Low Care 1"
ordine	intero		ordine di visualizzazione
utente			
id	PK intero		
username	testo(50)	univoco (senza distinzione maiuscole)	
nome, cognome	testo(80)		
ruolo	testo(12)	ADMIN MEDICO INFERMIERE OSS AMMIN	
password_hash	testo(255)		
attivo	booleano	predefinito vero	
deve_cambiare_password	booleano	predefinito vero	
tentativi_falliti	intero	predefinito 0	
bloccato_fino	data e ora, può essere vuoto		
ultimo_accesso	data e ora, può essere vuoto		
creato_il	data e ora		
creato_da	FK -> utente.id, può essere vuoto (primo admin)		
paziente			
id	PK intero		
reparto_id	FK -> reparto.id		
nome, cognome	testo(80)		
posto_letto	testo(10)		
gravita	testo(6)	nv verde giallo rosso (predefinito nv)	
data_arrivo	data		
data_dimissione_presunta	data, può essere vuota	(vincolo: >= data_arrivo)	
stato	testo(12)	RICOVERATO ELIMINATO	

```

creato_il / creato_da      data e ora / FK -> utente.id
eliminato_il / eliminato_da data e ora / FK -> utente.id, possono essere vuoti
INDICE UNIVOCO (reparto_id, posto_letto) solo dove stato = 'RICOVERATO'
INDICE (stato, reparto_id, data_dimissione_presunta)

nota
id                          PK intero
paziente_id                 FK -> paziente.id
autore_id                   FK -> utente.id
autore_nome                 testo(170)  copia "Nome Cognome" al momento della scrittura
autore_ruolo                 testo(12)   copia del ruolo al momento della scrittura
testo                       testo lungo (max 2000 caratteri, controllato dall'app)
data_nota                   data
creata_il                   data e ora
modificata_il               data e ora, può essere vuota
eliminata                   booleano    predefinito falso
eliminata_il / eliminata_da data e ora / FK -> utente.id, possono essere vuoti
INDICE (paziente_id, eliminata, data_nota, id)

nota_versione               (storico dei testi sostituiti da una modifica)
id                          PK intero
nota_id                     FK -> nota.id
testo_precedente            testo lungo
sostituito_il               data e ora
sostituito_da               FK -> utente.id

voce_checklist
codice                      PK testo(20)
testo                       testo(120)
ordine                      intero
attiva                      booleano    predefinito vero

checklist_spunta            (una riga = una voce spuntata per un paziente)
paziente_id                 FK -> paziente.id    } chiave primaria composta
voce_codice                  FK -> voce_checklist.codice }

evento_audit
id                          PK intero grande
quando                      data e ora
utente_id                   FK -> utente.id, può essere vuoto
username_tentato             testo(50), può essere vuoto (login falliti)
azione                      testo(30)
entita                      testo(20), può essere vuoto    es. paziente, nota, utente
entita_id                   intero, può essere vuoto
dettagli                    testo(255), può essere vuoto
indirizzo_ip                 testo(45)

```

La prima migrazione deve creare anche i dati fissi: i tre reparti e le 20 voci della checklist.

7. Requisiti non funzionali

- **Lingua e formati:** interfaccia interamente in italiano, date gg/mm/aaaa.
- **Browser:** funziona su Microsoft Edge e Google Chrome recenti; layout utilizzabile anche su schermi piccoli.
- **Sicurezza:**
 - protezione CSRF su tutte le richieste che modificano dati;
 - SECRET_KEY letta da .env;
 - cookie di sessione HttpOnly, SameSite=Lax e, in produzione, Secure;
 - output sempre con escape (autoescape di Jinja attivo);
 - accesso ai dati solo tramite ORM, mai SQL costruito concatenando testo;

- header di sicurezza: `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, e una `Content-Security-Policy` che consenta solo risorse del sito stesso;
- pagine di errore 403, 404 e 500 in italiano, senza dettagli tecnici; in produzione modalità debug disattivata.
- **Privacy (GDPR)**: nessun dato sanitario o testo di note nei log applicativi; niente codice fiscale, diagnosi o terapie tra i campi del paziente.
- **Log**: log applicativo su file con rotazione (per esempio 10 file da 5 MB).
- **Prestazioni**: la home si carica in meno di un secondo con 100 pazienti ricoverati.
- **Manutenibilità**: funzioni brevi, logica di business separata dalle route (per esempio in `servizi.py`), test automatici per le regole importanti.

8. Struttura del progetto

CruscottoDegenze/	
app/	
__init__.py	create_app(), registrazione blueprint ed estensioni
config.py	configurazioni Sviluppo / Test / Produzione
estensioni.py	db, migrate, login_manager, csrf
models.py	modelli SQLAlchemy (capitolo 6)
permessi.py	matrice ruoli -> permessi e decoratore
audit.py	funzione registra_evento(...)
comandi.py	flask crea-admin, flask dati-demo
auth/	login, logout, cambio password, profilo
pazienti/	home, inserimento, gravità, dimissione, eliminazione
agenda/	note e checklist (route JSON per i salvataggi immediati)
pdf/	generazione PDF con ReportLab
utenti/	gestione utenti e registro accessi
templates/	base.html + template per area + errori
static/	css/stile.css, js/app.js, font/DejaVuSans*.ttf
migrations/	generata da Flask-Migrate
tests/	test pytest
deploy/	installa.ps1, aggiorna.ps1, web.config, DEPLOY.md
.env.example	
requirements.txt	
wsgi.py	punto di ingresso per Waitress
README.md	

9. Fase 1 — Versione locale

Obiettivo: l'applicazione completa funziona sul PC Windows 11 di Micky con SQLite. Si avvia con `flask run` durante lo sviluppo e con `waitress-serve` per le prove con gli utenti.

Passi, ognuno con verifica e conferma prima del successivo:

1. **Scheletro**: struttura cartelle, ambiente virtuale, `requirements.txt`, `.env.example`, `git init` con `.gitignore`, pagina di prova "Il Cruscotto funziona".
2. **Database**:
 - modelli e migrazione iniziale con i dati fissi (reparti, voci della checklist);
 - comando `flask crea-admin`;
 - comando `flask dati-demo`, che carica gli stessi pazienti, utenti e note del prototipo e funziona solo in sviluppo.
3. **Accesso**: login, logout, cambio password obbligatorio, blocco dopo 5 tentativi, timeout di inattività, pagina "Il mio profilo".
4. **Home**: reparti con ordinamento, gravità, inserimento paziente, modifica della dimissione presunta, eliminazione logica.

5. **Agenda**: note con firma, raggruppamento per giorno, modifica con storico versioni, eliminazione logica.
6. **Checklist** dentro l'Agenda.
7. **PDF** del paziente.
8. **Gestione utenti e registro accessi**.
9. **Test automatici**, che devono verificare almeno:
 - i permessi di ogni ruolo su ogni route, con la risposta 403 attesa;
 - l'ordinamento per dimissione;
 - il controllo del letto occupato;
 - che solo l'autore possa modificare o eliminare una nota;
 - le eliminazioni logiche;
 - il blocco dopo 5 tentativi falliti;
 - che il PDF venga generato e contenga il nome del paziente.

Poi rifinitura grafica fedele al prototipo.

10. **README** con installazione, avvio, comandi utili e credenziali demo.

Criteri di accettazione della Fase 1:

- tutte le funzioni del capitolo 5 sono presenti e i test passano;
- l'aspetto corrisponde al prototipo v5;
- il programma funziona senza connessione a internet;
- nessuna password in chiaro nel database o nel codice.

10. Fase 2 — Versione di rete su macchina virtuale

Obiettivo: la stessa applicazione, senza modifiche al codice ma solo alla configurazione, è raggiungibile da tutti i PC di reparto all'indirizzo **<https://degenze.cdcmisnav.local>**.

Ambiente di destinazione:

- **VM applicativa**:
 - Windows Server 2025 Standard, nome `degenze-misnav`, nel dominio `cdcmisnav.local`;
 - IP statico nella rete server 192.168.5.0/24, da assegnare con il consulente;
 - agente SentinelOne installato.
- **Applicazione**:
 - Python installato per tutti gli utenti, applicazione in `C:\Apps\CruscottoDegenze`;
 - **Waitress** in ascolto solo su `127.0.0.1:8000`, eseguito come **servizio Windows** tramite NSSM, con un account di servizio dedicato **non amministratore**.
- **IIS come reverse proxy**:
 - moduli URL Rewrite e Application Request Routing;
 - HTTPS sulla porta 443 con certificato emesso dalla CA interna `ca-misnav` per `degenze.cdcmisnav.local`;
 - la porta 80 reindirizza a HTTPS;
 - IIS inoltra gli header `X-Forwarded-For` e `X-Forwarded-Proto`, e Flask li legge con `ProxyFix`, così il registro accessi riporta il vero IP del PC.
- **DNS**: record A `degenze.cdcmisnav.local` → IP della VM, creato su `dc-misnav1`.
- **Database**:

- SQL Server sull'istanza di `sql-misnav` (192.168.5.216), database `CruscottoDegenze`;
- l'applicazione usa un login SQL dedicato con i soli ruoli `db_datareader` e `db_datawriter`;
- le migrazioni vengono eseguite da un login separato con permessi di modifica dello schema, usato solo durante installazione e aggiornamenti.

- **Firewall:**

- sulla VM il Windows Firewall accetta solo le porte 443 e 80 dalla rete interna;
- sul FortiGate serve una regola che consenta alle postazioni (192.168.0.0/24) di raggiungere la VM sulla 443, e alla VM di raggiungere `sql-misnav` sulla 1433.

Consegne della Fase 2:

- `deploy/installa.ps1`, commentato riga per riga: crea l'ambiente virtuale, installa le dipendenze, crea `.env`, esegue le migrazioni, registra il servizio con NSSM.
- `deploy/aggiorna.ps1`: backup del database, arresto del servizio, aggiornamento del codice, dipendenze, migrazioni, riavvio, e istruzioni per tornare indietro in caso di problemi.
- `deploy/web.config` per il reverse proxy IIS.
- `deploy/crea_database.sql`: crea il database e i due login. Deve anche negare al login dell'applicazione il DELETE sulle tabelle `paziente`, `nota`, `nota_versione` e `utente`, e sia UPDATE sia DELETE su `evento_audit`, così le regole "nessuna cancellazione fisica" e "registro in sola aggiunta" sono garantite anche dal database.
- `deploy/DEPLOY.md` con la guida passo passo:
 - prerequisiti (ODBC Driver 18, moduli IIS, NSSM);
 - richiesta del certificato alla CA interna;
 - record DNS;
 - creazione del database e dei login SQL;
 - verifica finale;
 - procedura di backup e ripristino.
- Prima della produzione, una **prova generale** su una VM di test (Windows Server Evaluation con SQL Server Express) sul PC di Micky.

Criteri di accettazione della Fase 2:

- da un PC di reparto nel dominio, `https://degenze.cdcmisnav.local` si apre senza avvisi di certificato;
- login, note, checklist e PDF funzionano da più PC contemporaneamente;
- dopo il riavvio della VM il servizio riparte da solo;
- il backup del database è stato ripristinato con successo almeno una volta in prova.

11. Cosa non fare

- Non usare CDN o servizi esterni e non inviare dati fuori dalla rete aziendale.
- Non integrare né interrogare ResMedica o altri gestionali.
- Non cancellare fisicamente pazienti, note o utenti.
- Non salvare password in chiaro, né nel database né nei log né nei file di configurazione versionati.
- Non implementare funzioni non elencate senza averle proposte e fatte approvare.

12. Primo messaggio atteso

Rispondi:

1. confermando di aver letto il documento e il prototipo;
2. elencando i dubbi o i punti che ritieni poco chiari;
3. descrivendo il **passo 1 della Fase 1** e cosa devo avere installato sul PC per iniziare.

Non scrivere codice finché non do conferma.